Microscopic Traffic Simulation

Special topics on model calibration

Alessandro Scalese

March 13, 2026

Contents

To do:

- **p. 5**: REFINE AFTER WRITING
- **p. 7**: insert figure: proportion of sensors with stat signif res
- **p. 7**: insert figure: average counts for excluded sensors
- **p. 9**: inserire primo plot rmsn
- **p. 9**: inserire plot rmsn relativo
- **p. 11**: insert plot: hyperparameter optimization results
- **p. 12**: insert plot: calibration results
- **p. 12**: insert plot: calibration with individual —
- **p. 12**: insert weights
- **p. 12**: insert actual values!!!
- **p. 13**: inserire plot comparison spsa pcspsa

1. Introduction

Traffic simulation models are invaluable tools in traffic planning and management. Traffic forecasts are one of the foremost information tools used by decision makers in transportation matters such as transportation policies and infrastructure investments, with significant impacts on communities and society as a whole.

In the early 2000s, (Flyvbjerg et al., 2003; 2005) extensively analyzed the economic and traffic impacts of infrastructure projects undertaken in the previous decades. Among their findings, they highlighted that more than 50% of all projects reported differences in the forecasted and actual traffic demand above 20%, and that over 90% of the projects incurred in some measure of cost escalation, meaning the final cost was higher than the initial estimates. To top this off, they also underline that these — do not demonstrate — trends in their analysis period (ranging from the 1930s to the end of the 20th century).

This alone already does not paint a favourable landscape: inaccurate models can lead to possibly wrong or at least not optimal choices in transportation policies and infrastructure design, whose cost is more often than not underestimated, leading to misallocations of public money.

One of the reasons behind the inaccuracy of traffic forecasts is the complexity of traffic modeling itself: a complete traffic model must account for several interacting components, from the microscopic variables that influence driving behavior, to the drivers of route choice and pathing, all the way up to the definition of the overarching traffic flows. Each of these sub-components can be modelled in different ways, with different complexities and parameters, and each layer introduces errors and indirections which contribute to the inaccuracy of the final model. Furthermore, the lack of accurate data presents another challenge: the OD trips, which define the flows across the network zones, are not known a priori, but either have to be modeled from socio-economic and topological variables (as done in the four-step model for traffic assignment (Dios Ortúzar & Willumsen, 2024)) or estimated from traffic measurements (data-driven approach). These measurements are taken from traffic sensors such as induction loops or cameras, and their coverage of city networks is often sparse and uneven, leading to an undetermined optimization problem.

In this context, calibration can then be defined as the systematic adjustment of the model's parameters to minimize the discrepancy between real world measurements and the model's predictions. In transport modeling, these parameters are generally categorized into two groups: supply and demand. Supply parameters describe the physical and operational characteristics of the transportation system, including network geometry, speed limits, signal timings, as well as the microscopic driving behavior parameters that dictate how vehicles interact with the infrastructure (e.g. the lane-changing logic parameters). Demand parameters instead define the general population behavior - i.e. the location and amount of trips, which are typically represented in the OD matrix.

In our case, we are going to use a microscopic traffic simulator, SUMO (Lopez et al., 2018), as our model, using its default driving behavior and transport supply parameters. This leaves the demand parameters - the OD matrix - to be calibrated.

As the simulator acts as a “black-box” function, meaning we can only control the inputs (the OD matrix) and observe the outputs (the sensors measurements), but we do not have an analytical form that describes the relationship between them, we are precluded from using common gradient descent optimization algorithms due to computational limitations. In fact, estimating the gradient using (for example) a Finite Differences approach would require us to compute $2N$ function evaluations (i.e. simulations), where N is the number of parameters, which quickly becomes infeasible as the network grows (as both the number of parameters and the computational requirements of the simulations typically grow with the network’s size). Therefore, in transport model calbration, the optimization algorithm choice often falls onto the Simultaneous Perturbation Stochastic Approximation algorithm, originally developed by (Spall, 1998), which only requires 2 function evaluations to estimate the local gradient.

This algorithm and one of its variants will be used in the first part of this study, which focuses on the calibration of a traffic model for the city of Aachen. The second part of the study focuses instead on some dynamic applications such as Public Transport control and prioritization.

Indeed, this also showcases one of the unique advantages of SUMO: the ability to use its TraCI (Traffic Control Interface) API for fine-grained control over the simulation state, which allows us to implement real-time measurements and adaptive control strategies.

To do: REFINE AFTER WRITING In the following sections, —: The following chapters are organized as follows: Section 2 describes the Aachen network and the statistical determination of simulation replications; Section 3 details the SPSA-based calibration methodology and results; finally, Section 4 presents the TraCI-based implementation of dynamic Public Transport strategies.

2. Model Calibration

We are tasked with calibrating the origin-demand matrix for the morning peak-hour period (8-9am) for a medium sized network from the city of Aachen, in Germany. The network has been divided in twenty-three TAZes (Traffic Assignment Zones), which define the origin and destination of all trips on the network. This subdivision results in a total of 529 calibrable parameters (OD pairs). The simulation data will be collected by 599 synthetic loop detectors (around 15% coverage), which record counts, speeds and densities on a variety of links across the network.

2.1. Task 1: Evaluating simulation replications

2.1.1. The white noise hypothesis

Our simulator of choice (SUMO with mesoscopic simulations enabled) is inherently stochastic (noisy): several components of the model, which are designed to replicate human driving behavior (lane-changing decision making, rerouting probability), introduce various levels of randomness which have to be accounted for when analysing the model's output. As a consequence of this stochasticity, the same input parameters (OD flows) can lead to different results, meaning a single simulation run is not representative of the average traffic state across the network. To account for this variability, we need to adopt a "white noise" hypothesis, meaning that we assume that the variability in simulation outputs (meaning the error between them and the true values) follows a normal distrbution with zero mean and no systematic bias.

Mathematically, we can express this as:

$$Y_i = \overline{Y} + e \tag{1}$$

where:
$\boldsymbol{Y_i}$ are the outputs of a single simulation run
$\boldsymbol{\overline{Y}}$ is the true expected value of the system
$\boldsymbol{e_i}$ represents the stochastic noise

Under the white noise assumption, e is assumed to be indipenently and identically distributed, justifying the use of the sample mean across multiple simulation replications as a reasonable estimator of the network state.

2.1.2. Evaluating the required replications number

In order to determine the number of samples required to get statistically significant outputs with a 95% confidence interval and a 10% tolerance for each link, we are going to use a collection of data from 100 previous simulation runs. We can compute this number by applying the formula:

$$n = \left[\frac{2t_{(\alpha/2, n-1)} \cdot \sigma}{W} \right]^2 \tag{2}$$

where:
n number of required simulations
t t-students statistic (for $\alpha/2$ significance, $n-1$ DOF)
$\boldsymbol{\sigma}$ standard deviation

W width of tolerance interval

This computation needs to be performed for each sensor, across the range of simulations we want to investigate. A sensor is considered to give statistically significant results if the resulting required number of simulation replications, n, is lower than or equal to the number of simulations for which the test was conducted.

We decided to test the statistical significance of the sensors for up to 30 smulation runs to evaluate the trade-off between significance of the results and processing time. In **To do:** insert figure: proportion of sensors with stat signif res, it can be seen that, for 15 simulation runs, over 90% of the sensors provide statistically significant outputs. Further increasing the number of simulations does not significantly increase the percentage of compliant sensors, at least not in a way that justifies the increased computational effort.

Furthermore, we also looked into the characteristics of the outputs of the sensors that did not meet the statistical significance requirement for 15 simulation runs: as shown in **To do:** insert figure: average counts for excluded sensors, the majority of the sensors not yielding statistically significant results are characterized by extremely low traffic volumes, which are inherently more sensitive to stochastic fluctuations.

Therefore, we chose to set the number of simulation replications to 15, and to exclude non-compliant sensors from the subsequent calibration process, in order to focus on the ones with the highest signal-to-noise ratio.

2.2. Task 2: Exploration in Goodness of Fit functions and input space

2.2.1. GoF exploration

In section 2.1.2, we briefly established that low traffic volumes are more sensitive to stochastic noise compared to higher ones. A similar pattern needs to be taken into account during the choice of a proper Goodness of Fit (GoF) function. A GoF is a function that quantifies the "distance" of our model's results from the true values. It is the main indicator on which the optimization process is based, as it is used to derive the objective function value and, consequently, the direction of the optimization step.

It follows that the choice of a suitable GoF function is of paramount importance to the overall success of the calibration process (Hellinga, 1998). Some common objective functions, like the Root Mean Square Error (RMSE) and the Mean Absolute Percentage Error, are widely used in the literature as GoF functions for a variety of optimization problems. However, they may not be well-suited for the application on traffic calibration problems.

MAPE

The definition of MAPE computes the percentage error for each measurement by dividing the error for the true value:

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^{N} \left| \overline{y}_i - \frac{y_{\text{sim},i}}{\overline{y}_i} \right| \tag{3}$$

where:

N number of observations (sensors)

$\overline{y_i}$ true value for sensor i
$y_{\text{sim},i}$ observed (simulated) value for sensor i

From the formula, it is evident that, for small flows, the value of the function can explode. This would result in a disproportionate importance given to secondary flows, which as we have seen are also the ones more sensitive to stochastic variability.

RMSE

The RMSE is defined as the square root of the mean square error:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\overline{y}_i - y_{\text{sim},i}\right)^2} \tag{4}$$

where N, $\overline{y}_i$ and $y_{\text{sim},i}$ are defined as in the previous section.

As the RMSE squares the individual errors before averaging, it penalizes larger errors more heavily than smaller ones, therefore tending to be dominated by the errors on high-flow links (Toledo & Koutsopoulos, 2004). Moreover, it is an absolute metric, meaning that it weighs errors of different significance (e.g. an error of 100 vehicles on an Autobahn and on a residential street) on the same scale.

RMSN

The Normalized Root Mean Square Error (RMSN or NRMSE) is computed by weighing the RMSE against the global mean of the observations:

$$\text{RMSN} = \frac{\sqrt{N \cdot \sum_{i=1}^{N}\left(\overline{y}_i - y_{\text{sim},i}\right)^2}}{\sum_{i=1}^{N} y_i} \tag{5}$$

In contrast with the previous options, RMSN effectively balances the aggregate error by the total observed volume, avoiding both instability on small flows and the dominance of potential outliers. On top of this, it also is a dimensionless value, which also allows for a fair comparison across networks and varying traffic conditions. It is then a robust objective function choice for traffic calibration problem.

GLS-based weighting schemes

While the error metrics presented so far treat all observations with equal structural weights, a Generalized Least Squares (GLS) framework allows to explicitly incorporate the uncertainty associated with the data in the objective function formulation, effectively weighing the measurements based on their reliability. (Cascetta et al., 1993) introduced a simple weighting scheme that makes use of the observations' variance-covariance matrix:

$$\text{GLS} = \left(Y_{\text{sim}} - \overline{Y}\right)^T V^{-1} \left(Y_{\text{sim}} - \overline{Y}\right) \tag{6}$$

where:
$\boldsymbol{Y_{\text{sim}}}$ observations vector
$\boldsymbol{\overline{Y}}$ true values vector

V^{-1} inverse of the variance-covariance matrix

Using this approach, the residuals are weighed individually by the inverse of their variance, down-weighing less reliable observations such as measurements from sensors on low-flow links, which as stated before exhibit higher relative variance and are more susceptible to the simulator's stochastic noise. In an optimization perspective, this ensures that the calibration process is guided by high quality data points (i.e. sensors with high signal-to-noise ratios), thus reducing the impact of the simulator's stochasticity on the calbration process.

This aligns with what we have done in terms of filtering out non statistically significant measurements: our approach can thus be seen as a simplification of a GLS weighting scheme, as we are removing measurements with high intrinsic variance from the GoF computation.

2.2.2. Input space exploration

After establishing the use of the RMSN as the primary GoF metric, we conducted a line search to explore the sensitivity of the simulation to global variations in the input demand. We evaluated the performance of the model under different initial OD matrix multipliers, ranging from 0.1 to 2.0 with a 0.1 step size by computing our chosen GoG metric (RMSN) for three measures of performance (MOPs): counts, speeds, and densities.

The primary goal of this initial search is to identify potential systematic biases in the initial (a priori) OD matrices (i.e. a substantial over- or under-estimation of the real OD values). This way, we can select a starting point that is closer to the —.

The results of the exploration reveal distinct sensitivities for the three MOPs, as can be seen in **To do:** inserire primo plot rmsn:

- For traffic counts, the minimum RMSN occurs at a multiplier of 0.8.
- For densities, the minimum RMSN occurs at a multiplier of 0.6.
- For speeds, the RMSN remained consistently low (around 10%) across the whole multiplier range, showing a distinctly low variability.

These initial results align with the findings of (Toledo & Koutsopoulos, 2004), who showed that different Measures of Performance for the same – may yield conflicting results.

To better compare the sensitivity of these metrics, given their different scales (especially speeds vs the others), we normalized the RMSN values by the minimimum ones across the multiplier range. This allows us to analyze the relative — of the metrics, highlighting the MOPs that are most responsive to demand changes. As shown in **To do:** inserire plot rmsn relativo, OD changes in the explored range only have a marginal impact on the speeds, while the U-shaped counts and densities curves highlight their higher sensitivity to demand changes.

2.3. Task 3: Calibration using SPSA

Following the preliminary input space exploration, we implemened the SPSA algorithm (Spall, 1998) to perform the calibration of the Origin-Destination matrix. The algorithm implementation is based on a slightly modified version of the standard SPSA, with magnitude-dependent perturbation scaling and a common random numbers strategy for variance reduction. All relevant model and simulation setups are explained in the subsections below.

2.3.1. The SPSA algorithm

The SPSA algorithm updates the parameter vector θ (in our case, the flattened OD matrix) iteratively usng a stochastic approximation of the gradient:

$$\theta_{k+1} = \theta_k - a_k \cdot \hat{g}_k(\theta_k) \tag{7}$$

The gradient $\hat{g}_k$ is estimated by perturbing all elements of θ_k simultaneously using a random vector Δ_k, drawn from a ± 1 Bernoulli distribution:

$$\hat{g}_k(\theta_k) = \frac{y(\theta_k + c_k\Delta_k) - (\theta_k - c_k\Delta_k)}{2c_k\Delta_k} \tag{8}$$

The algorithm's hyperparameters, a_k and c_k, control the step size and the perturbation magnitude, respectively, and are typically defined as follows:

$$a_k = \frac{a}{(k+1+A)^\alpha} \tag{9}$$

$$c_k = \frac{c}{(k+1)^\gamma} \tag{10}$$

where a, c, A, α and γ are tunable hyperparameters that influence the convergence properties of the algorithm. The choice of these hyperparameters is crucial for the performance of the algorithm, as they affect the balance between exploration and exploitation in the optimization process.

Modifications to the standard SPSA

In our implementation, we introduced three modifications to the standard SPSA algorithm:

1. Magnitude-dependent perturbation scaling: instead of using a constant perturbation magnitude c_k for each element of the parameter vector, we scale it based on the magnitude of the parameters themselves, which helps maintain a consistent level of relative perturbation across parameters of different scales.
2. Common random numbers strategy: to reduce the impact of the simulator's stochasticity on the gradient estimation, we use common random numbers for both the positive and negative perturbations of the parameter vector. This should help isolate the effect of the perturbations from the inherent noise in the simulation, potentially leading to faster convergence. Empirically, we observed that the approach did indeed reduce the oscillations in the optimization process, especially in early algorithm iterations.
3. Clipping of the parameter updates: to ensure non-negative OD flows and to prevent excessively large updates of the parameter vector, we implemented a clipping mechanism that bounds the updates in a $\pm 15\%$ range of the current parameter values.

Hyperparameter tuning

As stated before, the choice of the SPSA hyperparameters is crucial for the convergence and performance of the algorithm. While there are some general guidelines for setting these hyperparameters, which can be found in the literature (Spall, 1998) and provide an acceptable starting point, their optimal values can vay significantly depending on the specific characteristics of the

problem. In our case, while maintaining the standard theoretically optimal values for the decay parameters α and γ (0.602 and 0.101, respectively) and for the stability constant A (around 10% of the total number of iterations), we performed an automatic search for the optimal values of the initial step size a and perturbation magnitude c.

More specifically, we leveraged the Optuna Python package (REFS) and its implementation of the Tree-structured Parzen Estimator (TPE) algorithm to perform a Bayesian optimization of the hyperparameters, allowing us to efficiently explore the hyperparameter space. After defining the search space for a $(1, 150)$ and c $(0.01, 1)$, we ran multiple optimization trials whose results can be seen in **To do:** insert plot: hyperparameter optimization results. For each trial, we ran the SPSA algorithm for a fixed number of iterations (20) and evaluated the resulting RMSN (based on counts) at each iteration, using Optuna's pruning mechanism to stop unpromising trials early and focus computational resources on the most promising hyperparameter configurations.

The optimal values identified at the end of the optimization process were $a = 100$ and $c = 0.6$.

2.3.2. Simulation configuration

For the calibration process, each simulation evaluation simulated a one-hour period (8-9am), with a warm-up period of 30 minutes (7:30-8am) to allow the system to reach a steady state before collecting data for the GoF evaluation, and a cool-down period of 30 minutes (9-9:30am) to allow the system to dissipate any congestion caused by the high demand during the peak hour. This configuration allows us to ensure the system reaches a steady state before collecting sensor data.

Each simulation was replicated 15 times, and the entire calibration process was run for a total of 500 iterations.

Interestingly, while the initial input space exploration suggested an optimal global scale of 0.8, early tests revealed that initializing the SPSA with a multiplier of 0.9 led to faster and more stable convergence of the algorithm.

2.3.3. Objective function definition

As discussed in the previous sections, we chose the RMSN as our GoF metric for the calibration process. In particular, we tested various GoF definitions, which differ in the weights given to the different MoPs in the RMSN computation. This allows us to analyze the impact of the choice of the GoF definition on the calibration results. The general GoF metric was defined as the weighted sum of the three MOP's RMSNs:

$$\text{GoF} = w_c \cdot \text{RMSN}_{\text{counts}} + w_d \cdot \text{RMSN}_{\text{densities}} + w_d \cdot \text{RMSN}_{\text{speeds}} \tag{11}$$

The weight combinations tested were the following:

- $w_c = 0.40, w_d = 0.50, w_s = 0.1$
- $w_c = 0.55, w_d = 0.35, w_s = 0.1$
- $w_c = 0.65, w_d = 0.25, w_s = 0.1$

These combinations were chosen based on the results of the line search on the initial OD matrix, which showed that the speeds were relatively insensitive to changes in the demand, while counts and densities were more responsive. We thus decided to give a low weight to the speeds RMSN,

while exploring different weight combinations for the counts and densities RMSNs to analyze their impact on the calibration results.

2.3.4. Calibration results

The results of the calibration process for the three experiments are shown in **To do:** insert plot: calibration results. All three weight combinations show a sharp initial RMSN descent in the early iterations, regardless of the specific weight combination, which suggests that the algorithm is able to quickly identify a direction of improvement in the parameter space. After this initial phase, the RMSN continues to decrease more gradually, with some oscillations, which is expected given the stochastic nature of the simulator and the optimization process.

What is more interesting is looking at the individual MOPs values across the iterations. In the figure **To do:** insert plot: calibration with individual — displays the decomposition of the unweighted RMSN components alongside the global metric for the **To do:** insert weights setup. It can be seen that, while the global weighted error stabilizes, the individul components continue to exhibit some — trends: the algorithm seems to keep trading off minor errors between counts and densities, as dictated by their assigned weights, while the speeds remain relatively unperturbed.

Overall, the final results demonstrate a successful calbration of our model for the Aachen network: **To do:** insert actual values!!!

2.4. Task 4: Advanced calibration with PC-SPSA

As an extension of our calibration approach, we then proceeded to implement the Principal Component SPSA (PC-SPSA) algorithm, proposed by (Qurashi et al., 2020). The main benefit of this algorithm s the reduction in the dimensionality of the optimization problem, achieved by performing Principal Component Analysis (PCA) on the historical data to identify the most significant directions of variability in the parameter space. This allows to focus the optimization process on the “directions” of maximum variability, which can lead to faster convergence and better results (Qurashi et al., 2020).

2.4.1. PCA analysis

In order to conduct the PCA analysis required to identify the principal components of the parameter space, we generated an initial population on the three available historical OD matrices, corresponding to different morning hours (7-8am, 8-9am, 9-10am). Following the methodology in (Qurashi et al., 2020), we generated 25 perturbations of each matrix by applying random multipliers to the OD flows:

$$\mathrm{OD}_{\mathrm{perturbed}} = (0.65 + 0.2 \cdot \delta) \cdot \mathrm{OD}_{\mathrm{base}} \tag{12}$$

where δ is a random variable drawn from a normal distribution with zero mean and standard deviation $\sigma = \frac{1}{3}$. This way, we generated a total of 75 perturbed OD matrices, which were then flattened and used as input for the PCA analysis.

The original parameter space consisted of approximately 520 non-null OD pairs. By applying PCA to the generated population, and selecting the principal components that cumulatively

explained 95% of the total variance, we ended up with $K = 29$ components, significantly reducing the dimensionality of the optimization problem.

2.4.2. Algorithm adaptation and tuning

In the PC-SPSA framework, the stochastic perturbations are not applied directly to the original parameter vector (i.e. to the elements of the matrix), but to the scores (eigenvalues) of the selected principal components. After the perturbation, the updated components are then projected back into the original space to reconstruct a matrix that can be used as input for the simulation.

As both the geometry and scale of the problem are significantly different from the previous direct approach, we had to implement some minor modifications to the SPSA algorithm:

- The magnitude-dependent perturbation scaling was adapted to the new problem space by using a relative (percentage) perturbation of the component scores
- The clipping was adapted to work on the scores, ensuring that in the minimization step, the component scores would not explode etc

Due to this changes and the new problem definition, the previously calibrated SPSA gain parameters a and c were found to be not applicable anymore. We thus conducted a new hyperparameter tuning phase via Optuna, which yielded the new PC-SPSA parameters:

- $a = 2$
- $c = 0.1$

Additionally, the stability parameter A was also adjusted and set to 20, following the common recommendation of setting it to around 10% of the total number of iterations, which in this case was set to 200 as we were expecting faster convergence compared to the plain SPSA.

All other simulation (warm-up and cool-down periods, number of replications) and algorithm aspects (common random numbers for plus/minus perturbations) were kept the same as in the previous calibration approach to ensure a fair comparison between the two methods.

2.4.3. Calibration results and comparison with plain SPSA

The implementation of the PC-SPSA variant yielded significant advantages compared to the plain SPSA approach, especially in regards of computational time and convergence speed. While the standard SPSA required hundreds of iterations to reach a stable solution, the PC-SPSA was able to achieve similar results in a fraction of the iterations without compromising the final calibration quality, leading to a significant reduction in the overall computational time required for the calibration process.

In the figure **To do:** inserire plot comparison spsa pcspsa we can see that the weighted RMSN achieved by PC-SPSA matches the results of the standard SPSA, but with a much faster convergence. This is likely due to the fact that the PC-SPSA focuses the optimization process on the most significant directions of variability in the parameter space, allowing it to more efficiently navigate towards optimal solutions. Moreover, the dimensionality reduction achieved by PCA also helps to mitigate the impact of the simulator's stochasticity on the optimization process, as it effectively filters out some of the noise in the parameter space.

3. Dynamic applications

The implementation of dynamic dwell times more closely reflects reality compared to fixed stop times. in reality, dwell times can vary significantly between stops, due to varying number of people boarding / alighting the bus, and also due to some random human behavior (think for example about people running for the bus which then waits for them, or people almost missing their stop that request the driver to open the door for them).

However, dynamic dwell times also mean that the total travel time of the bus can vary significantly, making timetables inaccurate: this is one of the many reasons behind the — of digital timetables at modern bus stops, which can show a more accurate estimate of the bus ETA compared to fixed timetables which can only take into consideration an average of the historical data. On the environmental side, at least in our simulation, dynamic dwell times lead to a general reduction of the idle time of the bus, therefore causing a reduction in fuel consumption (and so of the emission of pollutants and co2). So: travel time generally down, reliability up

Request stops aim to reduce travel times (and idle times) even further as useless stops are skipped directly (while in the ddt scenarios we still stopped for around 15 seconds at each stop, even if nobody needed to board or alight). Should I talk about the technical difficulties of the implementation? E.g. understanding when to compute the number of people alighting, boarding etc

In general, time travel reduction due to sometimes skipping one of the stops -> the one with fewer requests. This should also lead to a reduction of fuel consumption and emisisons compared to ddt -> understand why it is not shown in the graphs

What I need to do is run the simulation maybe like 4-5 times and then average the results Violin plots should be perfect for this

Public transport prioritization: application to a single junction in the whole network -> if this has good results, think about what would happen with multiple implementations across the network.

Analyze the base program to understand the available phases and how we can play around with them:

- phase bus
- phase “major” -> should probably call sidestream or something
- phase “major-left” -> people that need to turn
- interstage

We place a detector far enough (considering that there is a bus stop before the tl -> bad design) and we try to account for its stop time (V2I hypothesis). After the bus decides if it is going to stop or not, than we apply the logic:

- if phase bus
  - if the bus makes it before the end of the max green, do nothing
  - otherwise,
    - if we have enough time to run a quick cycle (min green times etc) -> do that
    - otherwise, extend green time
- otherwise,
  - if phase is not major

  - wait
- otherwise,
  - try to extend the major phase as long as possible to avoid disrupting the flow and only change when the bus is approaching 8allowing for a bit of time to let the queue dissipate

Instead of a simple “green time compensation” mechanism, we are going to implement a system that brings the tl back in sync with the adjacent one -> as we found empirically that prioritization would cause spillover and queues there due to people unable to occupy the junction in time. This is done by extending the major phase time and reducing the duration of the bus phase to bring them back in sync.

Very good results: significant reduction in travel times already with just one prioritized traffic light. Good reason to try and implement this in other junctions and for other bus lines as well, good pilot project.

Bibliography

Cascetta, E., Inaudi, D., & Marquis, G. (1993). Dynamic Estimators of Origin-Destination Matrices Using Traffic Counts. *Transportation Science*, *27*(4), 363–373. https://doi.org/10.1287/trsc.27.4.363

Dios Ortúzar, J. de, & Willumsen, L. G. (2024). *Modelling transport*. John wiley & sons.

Flyvbjerg, B., Skamris Holm, M. K., & Buhl, S. L. (2003). How common and how large are cost overruns in transport infrastructure projects?. *Transport Reviews*, *23*(1), 71–88. https://doi.org/10.1080/01441640309904

Flyvbjerg, B., Skamris Holm, M. K., & Buhl, S. L. (2005). How (In)accurate Are Demand Forecasts in Public Works Projects?: The Case of Transportation. *Journal of the American Planning Association*, *71*(2), 131–146. https://doi.org/10.1080/01944360508976688

Hellinga, B. R. (1998). Requirements for the calibration of traffic simulation models. *Proceedings of the Canadian Society for Civil Engineering*, *4*, 211–222.

Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., & Wiessner, E. (2018). Microscopic Traffic Simulation using SUMO. *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2575–2582. https://doi.org/10.1109/ITSC.2018.8569938

Qurashi, M., Ma, T., Chaniotakis, E., & Antoniou, C. (2020). PC–SPSA: Employing Dimensionality Reduction to Limit SPSA Search Noise in DTA Model Calibration. *IEEE Transactions on Intelligent Transportation Systems*, *21*(4), 1635–1645. https://doi.org/10.1109/TITS.2019.2915273

Spall, J. C. (1998). An overview of the simultaneous perturbation method for efficient optimization. *Johns Hopkins Apl Technical Digest*, *19*(4), 482–492.

Toledo, T., & Koutsopoulos, H. N. (2004). Statistical Validation of Traffic Simulation Models. *Transportation Research Record*, *1876*(1), 142–150. https://doi.org/10.3141/1876-15